School of Computer Sciences

CPC351/CPM351 Principles of Data Analytics

Academic Session: Semester 1, 2026/2027

**Assignment 01 – Basic of R Programming**

# I. Questions

You are required to answer the following questions:

1. You are provided with six text files containing the registration lists for six distinct courses. The department needs to create an exam timetable for these courses. Your task is to use R to generate an exam schedule that ensures no student has two exams scheduled in the same time slot.

| | Venue 1 (Room Capacity = 40) | Venue 2 (Room Capacity = 70) |
|---|---|---|
| Day 1 (Morning) | Slot 1 | Slot 2 |
| Day 1 (Afternoon) | Slot 3 | Slot 4 |
| Day 2 (Morning) | Slot 5 | Slot 6 |
| Day 2 (Afternoon) | Slot 7 | Slot 8 |

   a. Data cleaning and pre-processing: You are required to make a judgement call on how to clean the data.
   b. Conflict matrix: Write an R script to generate a 6 X 6 adjacency matrix that displays the exact number of overlapping students between every possible pair of courses.
   c. Timetable generation: Based on your conflict matrix, write an R script to assign each course to a specific slot (e.g., Slot 1, Slot 2, etc.). Your algorithm must ensure that there is no concurrent examination for any student. Is the eight-slot timetable sufficient for these six courses? Provide brief justification for your answer. If not, use your R script to generate and display a workable schedule.
   d. Back-to-back examination constraint: Improve your R script to include a new constraint: no student should have to sit for back-to-back examinations on the same day. Output your updated timetable. Explain in plain English the specific logic your R script used to check for and resolve back-to-back conflicts.

2. Ahmad needs to complete a Hamiltonian tour across ten specific delivery coordinates. The coordinates of the ten points are as follows:

| | | |
|---|---|---|
| P1 (60, 200) | P4 (140, 180) | P7 (200, 160) |
| P2 (180, 200) | P5 (20, 160) | P8 (140, 140) |
| P3 (80, 180) | P6 (100, 160) | P9 (40, 120) |
| | | P10 (100, 120) |

Instead of using straight-line Euclidean distance, Ahmad's travel costs are dictated by dynamic traffic and weather conditions. You will not be provided with a dataset for this question. Assume the unseen grading dataset (`ahmad_travel_logs.csv`) perfectly adheres to the following schema.

- `Origin_X` (Numeric)
- `Origin_Y` (Numeric)
- `Dest_X` (Numeric)
- `Dest_Y` (Numeric)
- `Weather` (Character string: e.g., 'Clear', 'Rainy', 'Storm')
- `Time_of_Day` (Character string: e.g., 'Morning', 'Afternoon', 'Evening')
- `Travel_Time_Minutes` (Numerical values represent historical travel time. Severe road closures are recorded as either `NA` or `-1`).

The features of `Origin_X`, `Origin_Y`, `Dest_X`, and `Dest_Y` contain the respective x and y coordinates of the ten points. Perform the following tasks:

a. Data pre-processing and cost matrix generation: Ahmad is scheduling a tour for a 'Rainy' morning. Write an R script to construct a 10 X 10 adjacency cost matrix representing the 75th percentile `Travel_Time_Minutes` for every possible pair of coordinates under these specific conditions. You must independently evaluate the integrity of the `Travel_Time_Minutes` column to identify and handle anomalies, such as records indicating severe road closures. Apply appropriate data cleaning treatments to remove these invalid records before aggregating your data. Furthermore, if your analysis reveals that a specific route is entirely blocked by road closures under these weather conditions (leaving no valid travel history), your final matrix must assign that route a severe penalty cost to ensure the routing algorithm avoids it.

b. Algorithm execution: Apply the Nearest Neighbour algorithm using your custom data-driven matrix (you may start from any point, P1 to P10). Output the final route and the total expected travel time.

Your R script must be generalised and robust. During the grading interview, your code will be tested on a new, unseen dataset with the exact same structure as explained above to ensure it automatically handles the data cleaning and routing without manual intervention. Do not hardcode solutions. If your script relies on hardcoded row indices or manual overrides, it will crash on the hidden dataset and result in a failing grade. You must be prepared to verbally explain your R logic, particularly how your algorithm handles the impassable routes penalty and the nearest neighbour selection. Because you are coding "blind," it is highly recommended that you generate a small dummy dataset in R to test your functions locally before submitting your final R script.

# III. Submission

This is a group assignment (a group of four members). The member grouping must be done via eLearn@USM.

You are required to submit a zip/rar package which consists of the following items to the eLearn@USM:

- R script (in .R format).
- An assignment report not more than 12 pages (in pdf format). Only the sample output screen shots and relevant explanation/write-up/description are expected. Do not paste your entire R code into the report, as it is evaluated via your submitted .R file. Also, a cover page which contains your details must be included in your assignment report.

The zip/rar package must be named according to the following notation: CPC351_[GroupNumber]_A01. For example, for Group03, they must name the zip/rar package as CPC351_Group03_A01.

One of the group members is required to submit the zip/rar package. Kindly communicate with your group member before the submission to avoid any miscommunication.

The submission deadline is 15 Nov 2026, (Sunday), 11:59 p.m. Failure to submit the assignment by the deadline will result in a Fail (0) for the assignment.

All the group members are required to reserve a time slot in advance to attend the upcoming code demonstration and grading interview. Each group must provide a detailed walkthrough of the script's logic, implementation steps, and functionality. During the interview, the instructor reserves the right to direct specific questions to individual group members; all members must demonstrate a clear understanding of the submitted R script and logic. Further announcements regarding the interview schedules will be made via eLearn@USM.

# IV. Grading Rubric

This assignment will be graded according to the grading rubric as shown in Table 1. The total will be scaled to 7% of your overall grade.

Table 1: Assignment 01 grading rubric.

| | Good<br>(3) | Satisfactory<br>(2) | Poor<br>(1) | Fail<br>(0) |
|---|---|---|---|---|
| Question 1 (40%)<br><br>(Evaluates Data Cleaning, Conflict Matrix, Scheduling, and Explanation) | • Script executes flawlessly.<br>• Data inconsistencies are cleaned using logical, justified judgement calls.<br>• Timetable strictly adheres to all constraints.<br>• Report & Interview: Report clearly explains the conflict resolution logic in plain English. During the live interview, the student verbally defends their script, demonstrating deep independent understanding. | • Script executes but throws non-fatal warnings.<br>• Data is mostly cleaned, but some edge cases are missed.<br>• Timetable is mostly correct but violates 1-2 minor constraints.<br>• Report & Interview: Report explanation is adequate but relies heavily on code jargon. During the interview, the student struggles slightly to articulate the conceptual "why" behind their logic. | • Script fails to execute or requires manual fixing.<br>• Data cleaning is fundamentally flawed or missing.<br>• Timetable violates major constraints (e.g., students have concurrent exams).<br>• Report & Interview: Report lacks a coherent explanation. Students cannot verbally explain their logic during the interview, suggesting AI generation without comprehension. | • No submission, late submission, or no-show for the grading interview. |

| | | | | |
|---|---|---|---|---|
| Question 2 (60%)<br><br>(Evaluates Hidden Data Handling, Cost Matrix, Routing & Explanation) | • Script is completely generalized and processes the hidden grading dataset flawlessly (zero hardcoding).<br>• Accurately filters conditions, drops anomalies (NA/-1), calculates the 75th percentile, and perfectly applies the impassable route penalty.<br>• Nearest Neighbour algorithm outputs the correct sequence and time.<br>• Report & Interview: Report explains the logic clearly. Student verbally explains the penalty and routing steps flawlessly during the interview. | • Script executes on hidden data but uses somewhat fragile/inefficient logic.<br>• Matrix calculation has minor errors (e.g., calculating mean instead of 75th percentile) or misses some anomalies.<br>• Algorithm generates a route.<br>• Report & Interview: Report explanation lacks depth. Student's verbal explanation of routing decisions or anomaly handling lacks clarity during the interview. | • Script crashes on the hidden dataset because the student hardcoded row indices or manual overrides.<br>• Fails to identify anomalies or apply the matrix penalty correctly.<br>• Report & Interview: Report is incomplete. Students cannot verbally explain their R logic or routing decisions, indicating they did not write or understand the code. | • No submission, late submission, or no-show for the grading interview. |

## ~~END OF ASSIGNMENT 01~~